SRI LANKA CUSTOMS

Report Management System (RMS)

System Documentation

Architecture, Data Model, API Reference and Maintenance Guide

Field	Detail
Document title	Customs RMS — System Documentation
System	Document Workflow Management System (Customs RMS)
Version	1.0
Date	July 2026
Prepared by	Dinujaya Supun
Repository	github.com/DinujayaSupun/Customs-RMS
Status	Final
Audience	Developers and system maintainers

This document describes the internal design of the Customs RMS application. It is intended for the developers and administrators who will maintain and extend the system.

Table of Contents

If the entries below appear empty, open the document in Word, select the table and press F9 to update the field.

1. Introduction

1.1 Purpose

Customs RMS is an internal document workflow system for Sri Lanka Customs. It replaces the manual circulation of physical files with a tracked digital workflow: officers receive documents, add minutes, attach files, forward them to colleagues or teams, record approval decisions, and complete or reopen them — with every action captured in an audit trail.

This document explains how the system is built. It covers the architecture, the database schema, the security and permission model, the document workflow, and the complete HTTP API, together with the maintenance notes needed to operate and extend the system after handover.

1.2 Scope

The following are covered in this document:

- System architecture and the layering conventions used across the codebase.
- The complete relational data model (16 tables) and the Flyway migrations that produce it.
- The authentication and authorisation model, including all 40 application permissions.
- The document lifecycle: statuses, workflow actions, recipient handling, and group routing.
- The full REST API surface and the real-time WebSocket channel.
- Testing, deployment and day-to-day maintenance guidance.

Operational procedures that already have dedicated documents in the repository — production deployment, Docker usage and the release checklist — are summarised here and cross-referenced rather than duplicated. Section 12.4 lists those documents.

1.3 Definitions

Term	Meaning
Document	A record of a customs report/file moving through the workflow. The central entity of the system.
Movement	One recorded handoff or action on a document (create, forward, return, approve, and so on).
Minute / Remark	A dated note added to a document by an officer, forming a running commentary.
Report At	The officer currently responsible for a document — the only user who may act on it.
Recipient set	The TO / CC / BCC audience of a document at one point in its history.
Recipient group	A named team that a document can be forwarded to; any group admin may then act on it.
Visibility	PUBLIC or PRIVATE, controlling who outside the recipient list may read a document.
Permission	One of 40 named capabilities that can be granted per role and overridden per user.

2. System Overview

Customs RMS is a two-tier web application: a Vue 3 single-page frontend and a Spring Boot REST backend backed by MySQL. The two communicate over JSON/HTTPS, plus one WebSocket channel used to push real-time notifications.

2.1 Principal capabilities

Capability	Description
Authentication	JWT-based login with BCrypt password hashing and a per-IP brute-force throttle.
Permission matrix	40 granular permissions, configurable per role by an administrator and overridable per individual user.
Document lifecycle	Create, list, filter, view, edit, delete, plus PUBLIC/PRIVATE visibility control.
Workflow actions	Forward, return, approve, reject, issue (mark done) and reopen.
Multi-recipient routing	A primary "Report At" owner plus CC and BCC recipients, each with independent view, attachment and minute permissions.
Recipient groups	Forward to a named team; any group admin may act while other members remain CC-only.
Minutes	A per-document timeline of dated notes.
Attachments	Upload, download, versioning and soft-delete, stored outside the web root.
Undo Send	A configurable window in which a sender may recall a forward or return before the recipient opens it.
Auto-forward	A scheduler that reassigns documents a Director of Customs has not opened within a configured timeout.
Real-time notifications	WebSocket push for forwards, returns and live permission changes.
Audit log	Every action recorded, with filtering and CSV export.
User administration	Create, edit, activate, deactivate, reset password, merge duplicates, bulk operations and CSV export.

2.2 Deployment topology

In production the application is served behind a reverse proxy (Nginx is recommended). The proxy terminates TLS and routes by path:

```
Users --> Nginx (TLS) --> /      --> Vue static build (dist/)
                          /api/  --> Spring Boot backend :8080
                          /ws/   --> Spring Boot backend :8080
                                     |        (WebSocket upgrade)
                                     |--> MySQL 8
                                     '--> Upload directory
```

The frontend is a static build: it contains no server-side logic, and the backend URL is compiled into it at build time through the VITE_API_BASE_URL variable. The backend is a single self-contained JAR.

3. Technology Stack

Layer	Technology	Version
Backend framework	Spring Boot (Web MVC, Data JPA, Security, Validation, Actuator, WebSocket)	4.0.2
Language / runtime	Java	17
Build tool	Maven (via the bundled Maven Wrapper)	—
ORM	Hibernate via Spring Data JPA	7.2.x
Schema migration	Flyway	11.x
JWT library	JJWT (jjwt-api, jjwt-impl, jjwt-jackson)	0.12.6
Database (production)	MySQL	8.x
Database (tests)	H2, in MySQL compatibility mode	—
Frontend framework	Vue 3 (<script setup> SFCs)	3.5.x
Frontend build	Vite	7.3.x
Routing	Vue Router	4.6.x
HTTP client	axios	1.13.x
Styling	Tailwind CSS with PostCSS	4.2.x
Icons	lucide-vue-next	0.577.x
Backend testing	JUnit 5 with Spring Boot Test and MockMvc	—
Frontend unit testing	Vitest with jsdom	3.2.x
End-to-end testing	Playwright (Chromium)	1.55+
Containerisation	Docker and Docker Compose	—
CI	GitHub Actions	—

All database access is written in portable JPQL rather than native SQL, so the persistence layer is not tied to MySQL. This is what allows the integration test suite to run against in-memory H2 while production runs on MySQL. A PostgreSQL driver is also present on the classpath.

4. Architecture

4.1 Request flow

Almost every user action follows the same path through the system. Understanding this one chain is enough to navigate the whole codebase:

```
Vue page --> API wrapper --> Controller --> Service --> Repository --> Table
```

For example, opening the document list executes:

```
pages/DocumentsPage.vue
  --> api/documents.api.js
        --> controller/DocumentController.java
              --> service/impl/DocumentServiceImpl.java
                    --> repository/DocumentRepository.java
                          --> documents table
```

4.2 Layer responsibilities

These boundaries are enforced by convention throughout the codebase and should be preserved when adding features:

Layer	Responsibility
Pages (Vue)	Screen state, forms and user interaction. No business rules.
API wrappers	HTTP paths and request/response handling. One wrapper module per domain area.
Controllers	Thin HTTP adapters: routing, resolving the authenticated user, and request validation.
Services	All business rules — permission checks, workflow transitions, audit logging and file handling.
Repositories	Database reads and writes, expressed in portable JPQL.
DTOs	The shape of data sent to the frontend. JPA entities are never exposed directly through the API.

Security is enforced in the service layer, not the UI. The frontend hides controls the user cannot use, but every permission is re-checked server-side, so a crafted API call cannot bypass a restriction that the interface merely hides.

4.3 Module map

The table below maps each functional area to its implementing files, which is the fastest way to locate the code behind a screen. The tables each area writes to are listed in Section 5.

Area	Frontend	Controller	Service
Login & profile	LoginPage.vue, ProfilePage.vue	AuthController	Spring Security, FileStorageService
Documents	DocumentsPage.vue, DocumentDetailsPage.vue	DocumentController	DocumentServiceImpl

Area	Frontend	Controller	Service
Inbox & workflow	InboxPage.vue	DocumentController	DocumentServiceImpl, DocumentRecipientServiceImpl
Minutes	DocumentDetailsPage.vue	DocumentRemarkController	—
Movement history	DocumentDetailsPage.vue	DocumentMovementController	—
Attachments	DocumentDetailsPage.vue	DocumentAttachmentController	AttachmentServiceImpl, FileStorageService
Recipient groups	GroupsPage.vue	RecipientGroupController	RecipientGroupServiceImpl
Audit logs	LogsPage.vue	LogsController	AuditLogServiceImpl
User administration	UsersPage.vue	AdminUserController	AdminUserServiceImpl
Permissions	PermissionsPage.vue	AdminPermissionController	PermissionServiceImpl, DcAutoForwardConfigServiceImpl
Notifications	services/realtimeNotifications.js	NotificationWebSocketHandler	RealtimeNotificationService

4.4 Repository layout

```
Customs-RMS/
├── rms-backend/          Spring Boot REST API + WebSocket (incl. Dockerfile)
├── rms-frontend/         Vue 3 SPA (incl. Dockerfile + nginx.conf)
├── scripts/              Local development helper scripts
├── packaging/            Offline image bundle for air-gapped servers
├── .github/workflows/    CI pipeline
├── docker-compose.yml    Local Docker stack (db + backend + frontend)
├── docker-compose.prod.yml   Production-style Docker stack
└── *.md                  README, DEPLOYMENT, QUICKSTART-DOCKER, RELEASE_CHECKLIST
```

5. Data Model

The schema contains 16 application tables plus Flyway's own flyway_schema_history. All tables use InnoDB and the utf8mb4 character set. The sections below group them by purpose.

5.1 Entity relationships

```
roles --1:N--> role_permissions
  |
  '--1:N--> users --1:N--> user_permissions   (per-user overrides)
              |
              |--1:N--> documents             (created_by / current_owner)
              '--N:M--> recipient_group       (via recipient_group_member)

documents --1:N--> document_movements      (workflow history)
    |     --1:N--> document_remarks        (minutes)
    |     --1:N--> document_attachments    (versioned, soft-deleted)
    |     --1:N--> document_user_views     (read receipts)
    '     --1:N--> document_recipient_sets (one active set at a time)
                       '--1:N--> document_recipients  (TO / CC / BCC)
```

5.2 Identity and access tables

Table	Key columns	Notes
users	user_id (PK), username (unique), full_name, password_hash, email, phone, department, role_id (FK), is_active, is_deleted, profile_picture_path	BCrypt password hash. Soft-deleted via is_deleted; deactivation is separate from deletion.
roles	role_id (PK), role_name (unique)	Seven fixed roles, seeded at startup.
role_permissions	role_permission_id (PK), role_id (FK), permission_name, enabled; unique (role_id, permission_name)	The permission matrix. One row per role/permission pair.
user_permissions	user_permission_id (PK), user_id, permission_name, enabled; unique (user_id, permission_name)	Per-user override. enabled=1 grants, enabled=0 revokes. No row means the role value is inherited.

5.3 Document tables

Table	Key columns	Notes
documents	id (PK), ref_no (unique), title, company_name, received_date, priority, status, visibility, doc_type, created_by_user_id, current_owner_user_id, current_owner_group_id, dc_assigned_at, dc_viewed_at, issued_at, completed_at, is_deleted, version	The central entity. current_owner_user_id is the "Report At" officer. version is the optimistic-locking column.

Table	Key columns	Notes
document_movements	id (PK), document_id, action_type, from_user_id, to_user_id, to_group_id, action_by_user_id, action_at, forward_visibility	Immutable workflow history. to_group_id is set when a forward targeted a group.
document_remarks	remark_id (PK), document_id (FK), remark_text, remarked_by (FK), remarked_at	Minutes. Foreign keys to documents and users.
document_attachments	id (PK), document_id, file_name, file_path, version_no, is_latest, uploaded_by, uploaded_at, is_deleted	Versioned: re-uploading the same logical file adds a version and clears is_latest on the previous row. Files live outside the web root.
document_user_views	id (PK), document_id, user_id, viewed_at; unique (document_id, user_id)	Read receipts. Drives the unopened/opened inbox filter and the Undo Send window.

5.4 Recipient tables

Recipients are versioned rather than edited in place. Each forward, return or manual change creates a new recipient set and deactivates the previous one, so the audience at any past point remains reconstructable.

Table	Key columns	Notes
document_recipient_sets	id (PK), document_id, movement_id, reason, is_active, created_by_user_id, created_at	Exactly one active set per document. reason records why the set was created.
document_recipients	id (PK), document_id, recipient_set_id, user_id, recipient_type, added_by_user_id, added_at, removed_at	recipient_type is TO, CC or BCC.

Valid values for recipient_set_reason: CREATE, FORWARD, RETURN, UNDO_SEND, AUTO_FORWARD, REOPEN, MANUAL_UPDATE.

5.5 Recipient group tables

Table	Key columns	Notes
recipient_group	recipient_group_id (PK), name (unique), color, image_path, created_by_user_id, created_at, updated_at	A named team. Referenced internally by id so renaming is safe.
recipient_group_member	recipient_group_member_id (PK), group_id, user_id, is_admin; unique (group_id, user_id)	is_admin distinguishes admins (who may act on group-held documents) from members (CC only).

5.6 Configuration and audit tables

Table	Key columns	Notes
audit_logs	id (PK), entity_type, entity_id, action_type, message, details_json, performed_by_user_id, performed_at	Append-only. details_json holds a structured snapshot of the action.

Table	Key columns	Notes
dc_auto_forward_config	config_id (PK), enabled, timeout_minutes, forward_return_allowed_statuses, approve_reject_buttons_enabled, undo_send_* (7 columns), updated_at	A single configuration row (id 1) holding admin-tunable workflow settings.
dc_auto_forward_receiver	dc_auto_forward_receiver_id (PK), dc_user_id (unique), receiver_user_id	Maps each Director of Customs to their own auto-forward receiver. A DC with no row is skipped by the scheduler.

5.7 Schema migrations

Flyway owns the schema. Hibernate is configured to validate only (spring.jpa.hibernate.ddl-auto=validate) and never alters tables, so the entities and the migrated schema must always agree.

Version	File	Change
V1	V1__initial_schema.sql	Baseline: the original 12 tables.
V2	V2__add_document_type.sql	Adds documents.doc_type (Internal/External). Nullable, so legacy rows stay valid; new documents require it at the application layer.
V3	V3__create_user_permissions.sql	Adds the user_permissions table for per-user permission overrides.
V4	V4__create_dc_auto_forward_receiver.sql	Adds per-DC auto-forward receiver mapping, replacing the single global receiver.
V5	V5__create_recipient_groups.sql	Adds recipient_group and recipient_group_member, plus documents.current_owner_group_id.
V6	V6__add_movement_to_group.sql	Adds document_movements.to_group_id so the timeline can show the target group.

baseline-on-migrate is enabled so an existing database can adopt Flyway by treating V1 as its baseline. To change the schema, add a new V7 migration — never edit an applied one.

6. Security Model

6.1 Authentication

Login is username and password against a BCrypt hash. A successful login returns a signed JWT access token that the frontend attaches to every subsequent request as an Authorization: Bearer header. Token lifetime defaults to eight hours and is configurable through JWT_EXPIRATION_MS.

The relevant classes are in the security package:

Class	Responsibility
JwtService	Signs and validates JWTs.
JwtAuthFilter	Servlet filter that authenticates each request from its Bearer token.
CustomUserDetailsService	Loads the user record for authentication.
CurrentUserService	Resolves the authenticated user inside controllers and services.
LoginAttemptService	Per-IP brute-force throttle.
NotificationHandshakeInterceptor	Authenticates the WebSocket handshake.
RestAuthenticationEntryPoint / RestAccessDeniedHandler	Return JSON errors instead of HTML login redirects.

6.2 Brute-force protection

Repeated failed logins from one client IP are blocked at the application layer. After a configurable number of failures within a window the source is blocked temporarily; a successful login clears its counter. Defaults are ten attempts and a fifteen-minute block, tunable through LOGIN_MAX_FAILED_ATTEMPTS and LOGIN_BLOCK_MINUTES.

6.3 Scoped download tokens

Files rendered directly by the browser — profile pictures and attachments opened in an image tag, an iframe or a new tab — cannot carry an Authorization header. Rather than putting the main JWT in the URL, the system issues a short-lived token scoped to one resource:

- POST /api/auth/me/profile-picture-token
- POST /api/attachments/{attachmentId}/download-token

The returned URL carries a download_token valid for roughly 120 seconds and usable only for that single resource. This keeps the long-lived access token out of browser URLs, server logs and referrer headers. Scoped tokens are explicitly rejected if presented as Bearer access tokens.

6.4 Authorisation

Authorisation is permission-based rather than role-based. A role is only a convenient default: what the system actually checks is whether the acting user holds a named permission, resolved as follows:

```
effective permission  =  user_permissions override   (if a row exists)
                         otherwise role_permissions   (the role default)
```

An administrator edits the matrix from the Permissions page. Changes take effect immediately — affected users receive a WebSocket notification and the interface re-evaluates their capabilities without requiring a re-login.

Beyond holding a permission, most workflow actions additionally require that the document is currently assigned to the acting user (or, for a group-held document, that they are an admin of the holding group). Both conditions are always checked server-side.

6.5 Document visibility

Each document is PUBLIC or PRIVATE. Access is granted if any of the following holds:

- The user can act on the document (owner, or admin of the holding group).
- The user is an active TO, CC or BCC recipient with the corresponding view permission.
- The document is PUBLIC and the user holds VIEW_PUBLIC_DOCUMENT.
- The user created the document and holds VIEW_OWN_CREATED_DOCUMENTS.
- The document is PRIVATE, the user holds VIEW_PRIVATE_DOCUMENT, and they appear in the document's forward trail — directly, or through membership of a group the document was forwarded to.

An unknown or missing visibility value is treated as PRIVATE. This fail-closed default ensures a malformed record can never widen access.

6.6 CORS and WebSocket origins

Allowed browser origins are set through APP_CORS_ALLOWED_ORIGINS, a comma-separated list that governs both the HTTP API and the WebSocket endpoint. It defaults to localhost values for development and must be set to the real domain in production, or the browser will silently block requests.

6.7 Operational security notes

- **JWT_SECRET** — a Base64 secret of at least 256 bits. Application startup fails if it is missing. Use a different secret per environment.
- **Uploads** — stored in an external directory (APP_UPLOAD_DIR), never inside the web root, so files cannot be requested directly.
- **Seeding** — default users are created only under the local, dev and e2e profiles when APP_SEED_ENABLED=true. Production has no seeded accounts by design.
- **Secrets** — .env is git-ignored. Production credentials must be supplied as server environment variables, never committed.

7. Roles and Permissions

7.1 Roles

Seven roles model the Customs hierarchy. They are seeded automatically at startup.

Role	Meaning
ADMIN	System administrator — user and permission management
DC	Director of Customs
DDC	Deputy Director of Customs
SDDC	Senior Deputy Director of Customs
SC	Superintendent of Customs
ASC	Assistant Superintendent of Customs
PMA	Personal Management Assistant

7.2 The 40 permissions

Permissions are grouped below by the area they govern. Every one can be toggled per role, and overridden for an individual user.

Document lifecycle

Permission	Grants
CREATE_DOCUMENT	Create new documents
EDIT_DOCUMENT_DETAILS	Edit document metadata
DELETE_DOCUMENT	Delete own documents
DELETE_ANY_DOCUMENT	Delete any document, regardless of owner
CHANGE_DOCUMENT_VISIBILITY	Switch a document between PUBLIC and PRIVATE

Viewing

Permission	Grants
VIEW_PUBLIC_DOCUMENT	Read any PUBLIC document
VIEW_PRIVATE_DOCUMENT	Read PRIVATE documents the user is in the trail of
VIEW_OWN_CREATED_DOCUMENTS	Read documents the user created
VIEW_ALL_DOCUMENTS	Read every document in the system
VIEW_ALL_HISTORY	View the full movement history
VIEW_LOGS	Access the audit log page
VIEW_SENT_MESSAGES	Access the Sent view
VIEW_HIDDEN_RECIPIENTS	See BCC recipients that are otherwise concealed
VIEW_REMARKS_WHEN_NOT_REPORT_AT	Read minutes without being the assigned officer

Workflow actions

Permission	Grants
FORWARD_DOCUMENT	Forward a document
FORWARD_PUBLIC	Forward with PUBLIC visibility
FORWARD_PRIVATE	Forward with PRIVATE visibility
RETURN_DOCUMENT	Return a document to its sender
APPROVE_DOCUMENT	Approve
REJECT_DOCUMENT	Reject
ISSUE_DOCUMENT	Mark as done / issued
REOPEN_DOCUMENT	Reopen a finalised document
ADD_REMARK	Add minutes

Attachments and recipients

Permission	Grants
UPLOAD_ATTACHMENT	Upload files
DELETE_ATTACHMENT	Delete files
MANAGE_DOCUMENT_RECIPIENTS	Manage recipients of own documents
MANAGE_ANY_DOCUMENT_RECIPIENTS	Manage recipients of any document
MANAGE_GROUPS	Create and manage recipient groups

CC and BCC capabilities

These twelve permissions control what copied recipients may do. Each has a CC_ and a BCC_ variant:

Permission pair	Grants to that recipient class
CC_VIEW_DOCUMENT / BCC_VIEW_DOCUMENT	Read the document
CC_VIEW_ATTACHMENTS / BCC_VIEW_ATTACHMENTS	See attachments
CC_UPLOAD_ATTACHMENTS / BCC_UPLOAD_ATTACHMENTS	Upload attachments
CC_DELETE_OWN_ATTACHMENTS / BCC_DELETE_OWN_ATTACHMENTS	Delete attachments they uploaded
CC_VIEW_TIMELINE / BCC_VIEW_TIMELINE	View the movement timeline
CC_VIEW_MINUTES / BCC_VIEW_MINUTES	Read minutes

7.3 Seeded role defaults

The values below are what the seeder writes on first startup. They are only starting points — an administrator can change any of them at runtime from the Permissions page.

Role	Default capability set
ADMIN	All 40 permissions.

Role	Default capability set
DC	Full document lifecycle and workflow, including approve, reject, issue and reopen, plus VIEW_ALL_DOCUMENTS, VIEW_ALL_HISTORY and VIEW_LOGS. Does not receive the CC/BCC permission set.
DDC, SDDC, SC, ASC	A shared "workflow" set: view, edit, minute, forward, return, attachments, manage recipients, and the full CC/BCC capability set. No approve, reject, issue, reopen or create.
PMA	Create, edit, minute, forward, return, change visibility and attachments. No approve/reject/issue/reopen, no CC/BCC set, no log access.

Note that approval authority (APPROVE_DOCUMENT, REJECT_DOCUMENT, ISSUE_DOCUMENT, REOPEN_DOCUMENT) is by default granted only to DC and ADMIN, reflecting the decision-making hierarchy.

8. Document Workflow

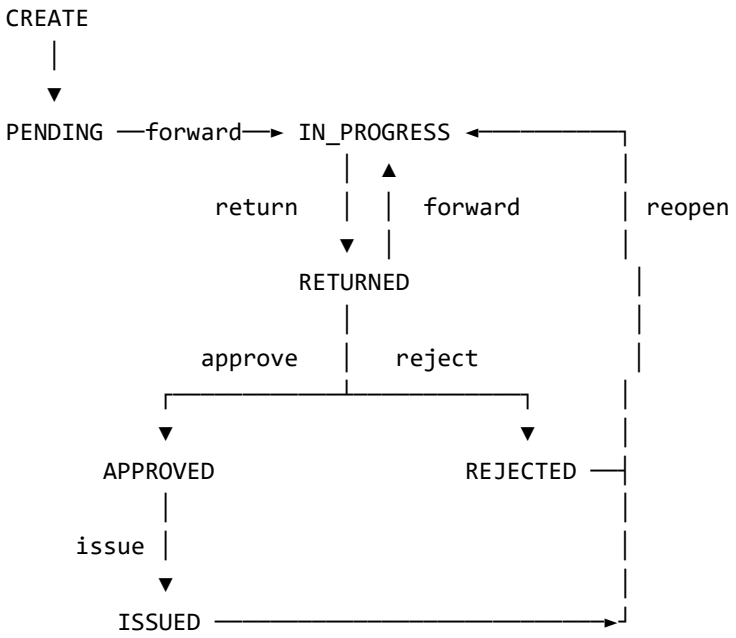
8.1 Statuses

Status	Meaning
PENDING	Newly created, not yet forwarded.
IN_PROGRESS	Actively moving between officers.
RETURNED	Sent back to a previous sender.
APPROVED	Approved; awaiting issue.
REJECTED	Rejected — a final state.
ISSUED	Completed and closed — a final state.

8.2 Movement types

Every action appends an immutable row to document_movements. The recorded action types are CREATE, UPDATE, FORWARD, RETURN, APPROVE, REJECT, ISSUE, REOPEN and UNDO_SEND.

8.3 Lifecycle



Rules governing transitions:

- Forward and return are blocked once a document is approved or in a final state.
- Issue is permitted only after approval.
- Reopen returns a finalised document to the active workflow. When approve/reject buttons are enabled, ISSUED is final and cannot be reopened; only APPROVED or REJECTED may be. Reopen always requires a reason.
- Approve and reject can be disabled system-wide by an administrator, which changes the available transitions.

8.4 Recipients

A forward addresses one primary recipient — who becomes the new "Report At" officer and the only person able to act — plus optional CC and BCC recipients who receive read-level access governed by the CC/BCC permissions.

Recipient sets are versioned rather than mutated: each forward, return, undo, auto-forward or manual change creates a new active set and deactivates the previous one. The audience at any past moment therefore remains reconstructable from history.

8.5 Recipient groups

A document may be forwarded to a group instead of an individual. The behaviour deliberately mirrors a messaging group:

- The group holds the document: documents.current_owner_group_id is set.
- Any admin of that group may act on it — forward, return, approve, minute and so on.
- current_owner_user_id still points at one group admin (the lowest user id) as a compatibility anchor, but authority comes from group membership, not from being that anchor.
- Remaining members are added as CC, so they can follow the document without being able to act on it.
- All group admins receive an "assigned" notification; plain members receive a "copied" notification.

Critical invariant: whenever ownership passes to a definite individual — forwarding to a person, returning, undoing a send, or auto-forwarding — current_owner_group_id must be cleared. If it is left set, every admin of the former group retains the ability to act on a document that no longer belongs to them. All four code paths currently clear it correctly; any new code that reassigns ownership must do the same.

8.6 Undo Send

A sender may recall a forward or return within a configurable window. The action is offered only when every one of the following holds:

- Undo Send is enabled by the administrator, and the action type is in the allowed list.
- The acting user is the original sender of that movement.
- The movement is the most recent one on the document, and the document has not moved on.
- The document is not finalised.
- The configured time window has not expired (24 hours by default).
- The recipient has not yet opened the document, when the "requires unopened" setting is active.

Undoing restores ownership to the sender, reinstates the previous recipient set, and records an UNDO_SEND movement so the recall itself remains visible in the history.

8.7 Automatic forwarding on timeout

A scheduled task reassigns documents that a Director of Customs has not opened within a configured timeout, preventing files from stalling. Its behaviour:

- Runs on a fixed delay, configurable through DC_AUTO_FORWARD_POLL_MS (60 seconds by default).
- Considers only documents assigned to a DC who has a receiver configured in dc_auto_forward_receiver; a DC without a mapping is skipped.
- Validates the receiver is still active and still holds a DDC or SDDC role before transferring.
- Preserves the existing CC and BCC recipients, records a FORWARD movement, writes an audit entry, and pushes the same real-time notification an interactive forward would.
- Processes documents in bounded batches so a single run cannot monopolise the database.

8.8 Real-time notifications

The frontend opens an authenticated WebSocket to /ws/notifications. Because browsers cannot set headers on a WebSocket handshake, the JWT is passed as a query parameter and validated by NotificationHandshakeInterceptor. The backend pushes events including DOCUMENT_FORWARDED, DOCUMENT_COPIED, DOCUMENT_RETURNED, undo notifications, and PERMISSIONS_UPDATED, which drive in-app toasts, browser notifications and live permission refreshes.

9. API Reference

All endpoints are JSON over HTTP and require an Authorization: Bearer <token> header unless noted. Server-side permission checks apply to every call; a request that passes validation may still be rejected on authorisation grounds.

9.1 Health

Method	Path	Description
GET	`/api/health`	Liveness check. Unauthenticated.

9.2 Authentication and profile

Method	Path	Description
POST	`/api/auth/login`	Authenticate; returns the access token, user details and effective permissions. Unauthenticated.
GET	`/api/auth/me`	Current user with effective permissions.
PUT	`/api/auth/me`	Update own profile details.
PATCH	`/api/auth/me/password`	Change own password.
POST	`/api/auth/me/profile-picture`	Upload a profile picture (multipart).
GET	`/api/auth/me/profile-picture`	Retrieve own profile picture.
POST	`/api/auth/me/profile-picture-token`	Issue a short-lived scoped token for browser rendering.
GET	`/api/auth/users`	List users available as workflow targets.

9.3 Documents

The base path /api/reports is retained as a legacy alias for /api/documents.

Method	Path	Description
POST	`/api/documents`	Create a document.
GET	`/api/documents`	List and filter documents (paged).
GET	`/api/documents/{id}`	Retrieve one document.
PUT	`/api/documents/{id}`	Update document details.
DELETE	`/api/documents/{id}`	Soft-delete a document.
GET	`/api/documents/my-inbox`	Documents assigned to or copying the current user (paged).
GET	`/api/documents/sent-messages`	Documents the current user has sent.
GET	`/api/documents/my-workload-stats`	Assigned, opened and unopened counts for the current user.
PUT	`/api/documents/{id}/recipients`	Replace the TO/CC/BCC recipient set.

9.4 Workflow actions

Method	Path	Description
POST	`/api/documents/{id}/forward`	Forward to a person (toUserId) or a group (toGroupId) — exactly one must be supplied.
POST	`/api/documents/{id}/return`	Return to the officer who last sent it.
POST	`/api/documents/{id}/undo-send`	Recall the most recent forward or return.
POST	`/api/documents/{id}/approve`	Approve.
POST	`/api/documents/{id}/reject`	Reject.
POST	`/api/documents/{id}/issue`	Mark as issued/done.
POST	`/api/documents/{id}/reopen`	Reopen a finalised document. Requires a reason.

9.5 Minutes, movements and per-document audit

Method	Path	Description
GET	`/api/documents/{documentId}/remarks`	List minutes.
POST	`/api/documents/{documentId}/remarks`	Add a minute.
PUT	`/api/documents/{documentId}/remarks/{remarkId}`	Edit a minute.
DELETE	`/api/documents/{documentId}/remarks/{remarkId}`	Delete a minute.
GET	`/api/documents/{documentId}/movements`	Full movement timeline, including any target group.
GET	`/api/documents/{documentId}/audit-logs`	Audit entries for one document.

9.6 Attachments

Method	Path	Description
POST	`/api/documents/{documentId}/attachments`	Upload a file (multipart).
GET	`/api/documents/{documentId}/attachments`	List a document's attachments.
GET	`/api/attachments/{attachmentId}/download`	Download a file.
POST	`/api/attachments/{attachmentId}/download-token`	Issue a short-lived scoped download token.
DELETE	`/api/attachments/{attachmentId}`	Soft-delete an attachment.

9.7 Recipient groups

Method	Path	Description
GET	`/api/groups`	List all groups with members and admin counts.
POST	`/api/groups`	Create a group. Requires MANAGE_GROUPS. The creator is always added as an admin.
PUT	`/api/groups/{id}`	Update name, colour and membership. At least one admin must remain.
DELETE	`/api/groups/{id}`	Delete a group. Blocked while it still holds active documents.

Method	Path	Description
GET	/api/groups/{id}/documents	Documents currently held by the group.

9.8 Audit log

Method	Path	Description
GET	/api/audit-logs	Search and filter the audit log (paged).
GET	/api/audit-logs/filter-options	Available filter values for the log UI.
GET	/api/audit-logs/export	Export the filtered log as CSV.

9.9 User administration

Method	Path	Description
GET	/api/admin/users	List users.
POST	/api/admin/users	Create a user.
PUT	/api/admin/users/{userId}	Update a user.
DELETE	/api/admin/users/{userId}	Soft-delete a user.
PATCH	/api/admin/users/{userId}/activate	Activate an account.
PATCH	/api/admin/users/{userId}/deactivate	Deactivate an account.
PATCH	/api/admin/users/{userId}/reset-password	Reset a user's password.
GET	/api/admin/users/{userId}/permissions	Read per-user permission overrides.
PUT	/api/admin/users/{userId}/permissions	Set per-user permission overrides.
GET	/api/admin/users/roles	List roles.
GET	/api/admin/users/duplicates	Detect likely duplicate accounts.
POST	/api/admin/users/merge	Merge duplicate accounts.
POST	/api/admin/users/bulk-deactivate	Deactivate several accounts at once.
POST	/api/admin/users/bulk-delete	Delete several accounts at once.
GET	/api/admin/users/export	Export users as CSV.

9.10 Permissions and workflow configuration

Method	Path	Description
GET	/api/admin/permissions	Read the full role/permission matrix.
PUT	/api/admin/permissions	Update the matrix.
PUT	/api/admin/permissions/page	Update a single page/section of the matrix.
GET	/api/admin/permissions/dc-auto-forward	Read auto-forward and Undo Send configuration.
PUT	/api/admin/permissions/dc-auto-forward	Update auto-forward and Undo Send configuration.
GET	/api/workflow-rules	Workflow rules the UI uses to decide which actions to offer.

9.11 WebSocket

Endpoint	Description
`/ws/notifications`	Authenticated push channel. The JWT is supplied as a query parameter because browsers cannot set WebSocket headers. Allowed origins come from APP_CORS_ALLOWED_ORIGINS.

10. Frontend Structure

The frontend is a Vue 3 single-page application built with Vite. Components use the <script setup> composition style. Business logic that benefits from unit testing is extracted into plain modules under src/utils, which is why the unit suite can cover workflow rules without mounting components.

10.1 Routes

Route	Page	Purpose
`/login`	LoginPage.vue	Authentication.
`/inbox`	InboxPage.vue	Message-style inbox with Received and Sent views.
`/documents`	DocumentsPage.vue	Searchable, filterable document list.
`/documents/new`	CreateDocumentPage.vue	Create a document.
`/documents/:id`	DocumentDetailsPage.vue	Document detail: metadata, workflow actions, minutes, attachments and timeline.
`/groups`	GroupsPage.vue	Manage recipient groups and view group-held documents.
`/profile`	ProfilePage.vue	Personal details, password and profile picture.
`/logs`	LogsPage.vue	Audit log with filtering and CSV export.
`/users`	UsersPage.vue	User administration.
`/permissions`	PermissionsPage.vue	Permission matrix and workflow configuration.

The /reports paths are kept as aliases of the corresponding /documents routes for backward compatibility. A catch-all route handles unknown paths.

10.2 Source layout

Directory	Contents
`src/pages`	One component per screen; owns screen state and user interaction.
`src/api`	axios wrappers, one module per domain area. All HTTP paths live here.
`src/utils`	Framework-free business logic — the main target of the unit test suite.
`src/router`	Route table and the authentication/permission navigation guards.
`src/auth`	Session storage and current-user helpers.
`src/services`	Cross-cutting services, including the WebSocket notification client.
`src/composables`	Reusable composition functions.
`src/layouts`	Application shell and navigation.
`e2e`	Playwright end-to-end specifications and helpers.

10.3 Route guarding

Navigation guards check both authentication and the permissions required by a route, redirecting unauthorised users away from restricted pages. This is a usability measure only — the same restrictions are independently enforced by the backend, so bypassing a guard grants no additional access.

11. Testing and Quality Assurance

11.1 Test suites

Suite	Scale	Command
Backend (JUnit 5, H2)	201 tests across 18 classes	`cd rms-backend; .\mvnw.cmd test`
Frontend unit (Vitest)	22 test files	`npm --prefix rms-frontend run test:unit`
End-to-end (Playwright)	19 tests	`cd rms-frontend; npm run test:e2e`

All three suites can be run in sequence with the helper script at the repository root:

```
.\run-all-tests.ps1 -DbPassword <your-mysql-password>
```

At the time of writing, all three suites pass in full.

11.2 Test strategy

- **Backend integration tests** — run against an in-memory H2 database in MySQL compatibility mode and exercise the real HTTP layer through MockMvc, so controllers, services, repositories and permission checks are covered together. No local MySQL is required.
- **Frontend unit tests** — cover the extracted logic modules in src/utils (workflow rules, inbox filtering, permission evaluation, formatting) without mounting components.
- **End-to-end tests** — drive a real Chromium browser against a live backend and a real database, covering login, document creation, forwarding, returning, approval, attachments, the audit log and profile management.

11.3 Continuous integration

The GitHub Actions workflow at .github/workflows/ci.yml runs on every push and pull request. It executes the backend suite against H2, then the frontend unit and Playwright suites against an ephemeral MySQL service container. Playwright reports and backend logs are uploaded as build artifacts, which is the first place to look when a CI run fails but local runs pass.

11.4 Notes for maintainers

- The E2E suite creates its own users and documents with recognisable prefixes and cleans them up afterwards, so it can safely run against a development database.
- The Playwright browser version must match the installed @playwright/test package. A mismatch produces immediate launch failures on every test rather than genuine assertion failures.
- E2E tests require the backend CORS configuration to permit the Playwright origin; otherwise every login step fails in a way that resembles an application fault.

12. Deployment and Operations

12.1 Environment variables

The backend reads configuration from a .env file locally, or from server environment variables in production.

Variable	Required	Purpose
DB_USERNAME	Yes	Database username.
DB_PASSWORD	Yes	Database password.
JWT_SECRET	Yes	Base64 signing secret (256-bit minimum). Startup fails if absent.
DB_URL	Optional	Full JDBC URL. Overrides the default MySQL connection.
APP_UPLOAD_DIR	Recommended	External directory for uploaded files.
APP_CORS_ALLOWED_ORIGINS	Production	Comma-separated allowed origins for both HTTP and WebSocket.
SPRING_PROFILES_ACTIVE	Optional	dev locally, prod in production.
JWT_EXPIRATION_MS	Optional	Token lifetime. Defaults to 8 hours.
DC_AUTO_FORWARD_POLL_MS	Optional	Auto-forward scheduler interval. Defaults to 60000.
LOGIN_MAX_FAILED_ATTEMPTS	Optional	Failed logins before an IP is blocked. Defaults to 10.
LOGIN_BLOCK_MINUTES	Optional	Block duration. Defaults to 15.
APP_SEED_ENABLED	Local only	Enables default user seeding. Must remain false in production.
APP_SEED_DEFAULT_PASSWORD	Local only	Password for seeded non-admin users.
APP_SEED_ADMIN_PASSWORD	Local only	Password for the seeded administrator.

The frontend is built with VITE_API_BASE_URL baked in at build time; it is not read at runtime, so changing the backend URL requires rebuilding the frontend.

12.2 Local development

Prerequisites: Java 17 or later, Node.js 20 or later, MySQL 8, and npm. Maven is not required separately — the repository includes the Maven Wrapper.

- Copy .env.example to .env and fill in the database credentials and a JWT secret.
- Run npm install once at the repository root.
- Run npm run dev:all to start the backend and frontend together.

Default addresses: the frontend on http://localhost:5173, the backend on http://localhost:8080. The dev:all script checks that both ports are free before starting and reports the owning process rather than terminating anything.

12.3 Production deployment

The application ships as two artifacts behind a reverse proxy: the Spring Boot JAR on port 8080, and the static Vite build served by the web server.

Three settings are the usual cause of a deployment that starts cleanly but does not work:

- **APP_CORS_ALLOWED_ORIGINS** — must list the real domain, and governs the WebSocket as well as the API.
- **VITE_API_BASE_URL** — must be set before building the frontend, not after.
- **SPRING_PROFILES_ACTIVE=prod** — switches Hibernate to schema validation and disables all seeding.

The reverse proxy must also forward WebSocket upgrade headers for the /ws/ path, or real-time notifications will silently fail while the rest of the application appears healthy.

Production has no seeded accounts by design; the first administrator must be created as described in the deployment guide.

12.4 Related documents

The repository contains dedicated operational documents that this section deliberately summarises rather than replaces:

Document	Covers
`README.md`	Feature overview, setup, architecture summary and local development.
`DEPLOYMENT.md`	Full production deployment: environment, database, systemd and Nginx configuration, first-admin bootstrap, hardening, rollback and troubleshooting.
`QUICKSTART-DOCKER.md`	Running the entire stack in Docker with one command.
`RELEASE_CHECKLIST.md`	Pre-release verification steps.
`packaging/README.md`	Offline image bundle for air-gapped servers.
`rms-backend/README.md`	Backend configuration and secrets handling.

13. Maintenance and Handover Notes

This section records the conventions and non-obvious behaviours that are most useful to whoever maintains the system next.

13.1 Adding a feature

New functionality normally touches the same chain of files. Using a new document action as the example:

- A control in the relevant Vue page.
- A function in the matching src/api wrapper.
- A route on the appropriate controller.
- Business rules and permission checks in the service layer.
- Repository access, written in portable JPQL.
- An audit log entry.
- Tests — an integration test for the backend rule, and unit tests for any extracted frontend logic.

13.2 Invariants to preserve

Invariant	Why it matters
Clear current_owner_group_id whenever ownership passes to an individual.	Otherwise admins of the former group keep the ability to act on a document that is no longer theirs. Applies to forward-to-person, return, undo send and auto-forward.
Enforce every permission in the service layer.	The UI only hides controls. A direct API call must not be able to bypass a restriction.
Treat unknown visibility as PRIVATE.	Fail-closed: a malformed record must never widen access.
Save minutes before changing ownership.	A minute added during a forward belongs to the sender, not the recipient.
Never expose entities directly from controllers.	DTOs keep the API contract stable and prevent accidental disclosure of internal fields.
Add a new Flyway migration; never edit an applied one.	Hibernate validates the schema at startup and will refuse to run if entities and tables disagree.
Keep repository queries in JPQL.	Preserves database portability, which the H2-based test suite depends on.

13.3 Behaviour that commonly surprises new maintainers

- **Recipients are versioned, not edited.** Changing recipients creates a new set and deactivates the old one, so the historical audience is preserved. Queries must therefore filter on the active set.
- **Group membership grants access indirectly.** A user may be able to see a document without appearing in any recipient row for it, because they belong to a group that holds or previously received it. Access queries need to account for both paths.
- **The group creator is always an admin.** On creation the backend re-adds the creator as an admin even if they were removed from the member list, to prevent a group existing with no one able to manage it. Editing the group afterwards can remove them, provided one admin remains.

- **current_owner_user_id is only an anchor for group-held documents.** It points at the lowest-id group admin for compatibility. Authority comes from group membership, not from matching this column.
- **Documents and users are soft-deleted.** Rows persist with is_deleted set. Queries must exclude them explicitly; the repository methods already do.
- **Approve and reject can be switched off system-wide.** When disabled, the available workflow transitions change, and reopen becomes valid from additional states.

13.4 Housekeeping

- An empty directory named "rms-frontend;C" exists in the repository root. It is an artifact of a mistyped shell command, contains nothing, and can safely be deleted.
- note.txt in the repository root holds informal quick-reference notes that predate the formal documentation. README.md is the authoritative setup reference.

13.5 Suggested future work

- Add OpenAPI/Swagger generation so the API reference in Section 9 stays synchronised with the code automatically.
- Extend end-to-end coverage to the recipient group workflows, which are currently verified by backend integration tests only.
- Consider archiving or partitioning audit_logs, which grows without bound in normal operation.
- Add database backup and restore procedures to the operational documentation.

Appendix A: Enumerated Values

Reference list of the fixed value sets used across the schema and API.

Enumeration	Values
Status	`PENDING, IN_PROGRESS, RETURNED, APPROVED, REJECTED, ISSUED`
Priority	`LOW, MEDIUM, HIGH, URGENT`
DocumentType	`INTERNAL, EXTERNAL`
Visibility	`PUBLIC, PRIVATE`
MovementActionType	`CREATE, UPDATE, FORWARD, RETURN, APPROVE, REJECT, ISSUE, REOPEN, UNDO_SEND`
RecipientType	`TO, CC, BCC`
RecipientSetReason	`CREATE, FORWARD, RETURN, UNDO_SEND, AUTO_FORWARD, REOPEN, MANUAL_UPDATE`
Roles	`ADMIN, DC, DDC, SDDC, SC, ASC, PMA`

Appendix B: Document Control

Version	Date	Author	Summary
1.0	July 2026	Dinujaya Supun	Initial system documentation covering architecture, data model, security, workflow, API reference and maintenance guidance.

End of document.